

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Volver al futuro: Incorporación de operadores de pasado al lenguaje lógico de MTSA

Tesis de Licenciatura en Ciencias de la Computación

Joaquín Laks

Director: Sebastián Uchitel

Buenos Aires, 2026

VOLVER AL FUTURO: INCORPORACIÓN DE OPERADORES DE PASADO AL LENGUAJE LÓGICO DE MTSA

.

Palabras claves: .

Índice general

1.. 1 1
1.1. El lenguaje pLTL 1

2.. Experimentación 3
2.1. Generación aleatoria de fórmulas 3
2.2. Evaluación temporal y del tamaño del autómata 3
2.3. Velocidad y correctitud de Model Checking 4

Bibliografía 5

1. 1

1.1. El lenguaje pLTL

La gramática de LTL implementada hasta el momento en MTSA es:

$$
\begin{array}{lll}
\pi^i \models \top & & \\
\pi^i \models \neg\varphi & \iff \pi^i \not\models \psi & \\
\pi^i \models \varphi \wedge \psi & \iff \pi^i \models \varphi \text{ y } \pi^i \models \psi & \\
\pi^i \models \varphi \vee \psi & \iff \pi^i \models \varphi \text{ o } \pi^i \models \psi & \\
\pi^i \models \bigcirc\varphi & \iff \pi^{i+1} \models \varphi & \textit{(Next, Siguiente)} \\
\pi^i \models \varphi\, \mathcal{U}\, \psi & \iff \exists k \geq i / \pi^k \models \psi \text{ y } \forall i \leq j < k, \pi^j \models \varphi & \textit{(Until, Hasta)} \\
\pi^i \models \varphi\, \mathcal{R}\, \psi & \iff \forall k \geq i, \pi^k \models \psi \text{ o } \exists i \leq j < k / \pi^j \models \varphi & \textit{(Release, Libera)}
\end{array}
$$

De estos símbolos se desprenden las definiciones:

$$
\begin{array}{lll}
\bot & \equiv \neg\top & \textit{(Falso)} \\
\Diamond\varphi & \equiv \top\, \mathcal{U}\, \varphi & \textit{(Eventualmente } \varphi\textit{)} \\
\Box\varphi & \equiv \neg\Diamond\neg\varphi & \textit{(Siempre } \varphi\textit{)} \\
\varphi\, \mathcal{W}\, \psi & \equiv \psi\, \mathcal{R}\, (\varphi \vee \psi) & \textit{(Until débil)}
\end{array}
$$

Estos últimos símbolos son parte del lenguaje LTL al que tiene acceso el usuario, pero el parser los transforma inmediatamente a su equivalente con los operadores de la primera tabla.

Kesten et al. [2] utiliza los operadores $\top, \neg, \wedge, \bigcirc, \mathcal{U}$, y agrega los operadores de pasado:

$$
\begin{array}{lll}
\pi^i \models \bullet\varphi & \iff i > 0 \text{ y } \pi^{i-1} \models \varphi & \textit{(Yesterday, Anterior)} \\
\pi^i \models \varphi\, \mathcal{S}\, \psi & \iff \exists k, 0 \leq k \leq i / \pi^k \models \psi \text{ y } \forall k \leq j < i, \pi^j \models \varphi & \textit{(Since, Desde)}
\end{array}
$$

De los cuales, de la misma forma que en futuro, por comodidad del usuario, agregamos las definiciones:

$$
\begin{array}{lll}
\blacklozenge\varphi & \equiv \top\, \mathcal{S}\, \varphi & \textit{(Eventualmente, en el pasado, } \varphi\textit{)} \\
\blacksquare\varphi & \equiv \neg\blacklozenge\neg\varphi & \textit{(Siempre, en el pasado, } \varphi\textit{)}
\end{array}
$$

Por simplicidad, MTSA mantiene siempre las fórmulas en forma normal negada, es decir que el $\neg$ solo puede estar antes de una proposición. Para poder mantener esta restricción, cuando se crea una fórmula con una negación, se aplica la transformación recursiva:

$$
\begin{array}{lll}
\neg X\varphi & \rightarrow \tilde{X}\neg\varphi & \text{Si } X \text{ es un operador unario y } \tilde{X} \text{ su dual} \\
\neg(\psi X\varphi) & \rightarrow \neg\psi\tilde{X}\neg\varphi & \text{Si } X \text{ es un operador binario y } \tilde{X} \text{ su dual} \\
\neg\varphi & \rightarrow \neg\varphi & \text{Si } \varphi \text{ es una proposición}
\end{array}
$$

Queda claro entonces que es necesario que todo operador tenga un dual. Para todos los operadores de futuro ya presentes, existe en el lenguaje un dual ($\mathcal{U}$ y $\mathcal{R}$ son duales, y $\bigcirc$ es su propio dual). Pero Kesten et al. [2] no presenta operadores duales para los símbolos de pasado que introduce.

Para el $\mathcal{S}$, agregamos al lenguaje su dual $\mathcal{T}$, definido como:

$$\left| \pi^i \models \varphi \mathcal{T} \psi \iff \forall 0 \leq k \leq i, \pi^k \models \psi \text{ o } \exists k < j \leq i / \pi^j \models \varphi \quad \textit{(Trigger, Gatilla)} \right.$$

A diferencia de su equivalente de futuro, no es verdad que $\bullet$ es su propio dual. Esto se debe a la condición agregada $i > 0$ que hace que $\pi^0 \not\models \bullet\varphi$ para cualuier π y φ. Entonces no es cierto que $\neg \bullet \varphi \equiv \bullet\neg\varphi$, porque en el momento 0, $\pi^0 \models \neg \bullet \varphi$ pero $\pi^0 \not\models \bullet\neg\varphi$. Por eso es necesario agregar el operador $\tilde{\bullet}$, con semántica:

$$\left| \pi^i \models \tilde{\bullet}\varphi \iff i = 0 \text{ o } \pi^{i-1} \models \varphi \quad \textit{(Yesterday débil)} \right.$$

Es interesante notar que, evaluados en cualquier momento que no es el inicial, $\bullet$ y $\tilde{\bullet}$ son equivalentes.

Agregando $\mathcal{T}$ y $\tilde{\bullet}$ a los operadores usados en Kesten et al. [2] y MTSA, obtenemos un lenguaje de lógica temporal con operadores de pasado y futuro que podemos mantener en forma normal negada.

2. EXPERIMENTACIÓN

El principal objetivo de la experimentación de este trabajo es comparar la eficiencia y utilidad del nuevo algoritmo implementado en relación al algoritmo preexistente en MTSA. Además de verificar la correctitud de la implementación. Para esto analizamos distintas métricas.

2.1. Generación aleatoria de fórmulas

Usamos una generación aleatoria de fórmulas inspirada en Daniele et al. [1] para usar en distintas experimentaciones. Para las fórmulas de puro futuro, se usa el conjunto de operadores $\{\neg, \bigcirc, \vee, \wedge, \mathcal{U}, \mathcal{W}\}$. Para las fórmulas con operadores de pasado se agregan los operadores $\{\bullet, \tilde{\bullet}, \mathcal{S}, \mathcal{T}\}$.

La generación de fórmulas es paramétrica respecto a variables N, L, y P. Se genera una fórmula de tamaño exáctamente L con un alfabeto de tamaño N con el siguiente procedimiento.

- Si $L = 1$, devolver una fórmula que consiste en un símbolo elegido uniformemente en el alfabeto.
- Si $L = 2$, elegir uniformemente al azar un operador op entre los unarios y un símbolo p en el alfabeto. Devolver $op(p)$.
- Si $L > 2$, se decide con probabilidad P si el próximo operador a elegir es temporal dual o no. Es decir, si se tiene el conjunto de operadores $\{\neg, \bigcirc, \bullet, \tilde{\bullet}, \vee, \wedge, \mathcal{U}, \mathcal{W}, \mathcal{S}, \mathcal{T}\}$ hay una probabilidad P de que op se elija uniformemente en $\{\mathcal{U}, \mathcal{W}, \mathcal{S}, \mathcal{T}\}$ y probabilidad $1 - P$ de elegirlo entre el resto de operadores. En caso de que el operador elegido sea unario, se crea recursivamente una fórmula al azar ψ de tamaño $L - 1$ y se devuelve $op(\psi)$. Si op es binario, se elige uniformemente un $1 \leq S \leq L - 2$, se generan dos fórmulas ψ_1, ψ_2 de tamaño S y $L - S - 1$ respectivamente, y se devuelve $op(\psi_1, \psi_2)$.

Se puede observar que cuando P es igual a la proporción de operadores temporales duales en el conjunto, los operadores se eligen uniformemente.

Dado unos parámetros N, L, y P, y un conjunto de operadores, se generan F fórmulas distintas para obtener un conjunto de fórmulas a evaluar.

2.2. Evaluación temporal y del tamaño del autómata

Seguimos la experimentación de Daniele et al. [1] con el objetivo de analizar las diferencias de tiempo de ejecución y de tamaño de autómata entre distintos algoritmos. Para eso generamos conjuntos de fórmulas para distintos valores de N, L, P con $F = 100$, a cada conjunto lo llamamos $\Phi_{N,L,P}$.

Para comparar la variable θ (por ejemplo, cantidad de aristas, o tiempo de ejecución) entre dos algoritmos x e y, generamos un $\Phi_{N,L,P}$ y, para cada $\varphi \in \Phi_{N,L,P}$, generamos el autómata para $\neg\varphi$ usando el algoritmo x, después guardamos el promedio de todos los θ de esa generación en θ_x. Hacemos lo mismo para generar θ_y.

Una vez calculados θ_x y θ_y calculamos $\frac{\theta_y}{\theta_x}$ como medida de "cuánto mejor es el algoritmo x sobre el y en el parámetro θ". Como medimos parámetros que queremos minimizar, un valor

mayor a 1 de $\frac{\theta_y}{\theta_x}$ indica que x presenta una mejora sobre y, mientras que un valor menor a 1 indica lo contrario.

Dados dos algoritmos y un parámetro a medir, exploramos el espacio de posibilidades de N, L, P y hacemos esta medición para cada valor.

Particularmente nos interesa analizar la diferencia en tiempo de ejecución, cantidad de estados, y cantidad de aristas entre el algoritmo ya implementado en MTSA y esta nueva implementación. Como el trabajo de esta tesis no es una optimización sino una extensión del lenguaje lógico, no esperamos mejoras en ninguno de estos parámetros. Más bien buscamos determinar la magnitud de sus diferencias. Para esta evaluacion no usamos operadores de pasado para poder comparar la misma fórmula en los dos algoritmos.

También nos interesa analizar la magnitud de las optimizaciones presentadas sobre herencia selectiva de aristas en Kesten et al. [2]. Para eso, medimos la cantidad de aristas en el algoritmo con estas optimizaciones y sin ellas.

Por último, queremos caracterizar la mejora que trajo la programación dinámica en la corrección de aristas. Como el autómata resultante es el mismo, analizamos solamente el tiempo de ejecución del algoritmo con la memoización y sin la memoización.

Para estos últimos dos experimentos sí usamos los operadores de pasado.

2.3. Velocidad y correctitud de Model Checking

Para analizar la diferencia de velocidad del model checking y verificar la correctitud de los contraejemplos, usamos la familia `crscounter`. Para un $N \in \mathbb{N}$, generamos un LTS

sadfas
asdf

Bibliografía

[1] Marco Daniele, Fausto Giunchiglia, and Moshe Y. Vardi. Improved automata generation for linear temporal logic. In Nicolas Halbwachs and Doron Peled, editors, *Computer Aided Verification*, pages 249–260, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg. ISBN 978-3-540-48683-1.

[2] Y. Kesten, Z. Manna, H. McGuire, and A. Pnueli. A decision algorithm for full propositional temporal logic. In Costas Courcoubetis, editor, *Computer Aided Verification*, pages 97–109, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg. ISBN 978-3-540-47787-7.